# Parallel & Distributed Computing

## REAL-WORLD ENVIRONMENTS THAT EXHIBIT PARALLELISM

*“THE FUTURE IS PARALLEL.” – NVIDIA*

**Dr. Muzamil Ahmed** (Assistant Professor)
Namal University Mianwali

# Lesson from Quran



# Google Scholar Profile



## Dr. Muzamil Ahmed

FOLLOW

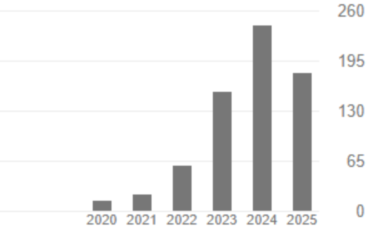
Assistant Professor Computer Science, Department of Computer Science, Namal University Pakistan
Verified email at namal.edu.pk

Data Science · Data Mining · Machine Learning · Information Retrieval · Deep Learning

| TITLE | CITED BY | YEAR |
|---|---|---|
| Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms<br>FK Alarfaj, I Malik, HU Khan, N Almusallam, M Ramzan, M Ahmed<br>Ieee Access 10, 39700-39715 | 375 | 2022 |
| A review on state-of-the-art violence detection techniques<br>M Ramzan, A Abid, HU Khan, SM Awan, A Ismail, M Ahmed, M Ilyas, ...<br>IEEE Access 7, 107560-107575 | 183 | 2019 |
| Real-Time Violent Action Recognition Using Key Frames Extraction and Deep Learning<br>M Ahmed, M Ramzan, HU Khan, S Iqbal, M Attique<br>Computers, Materials & Continua 69 (2), 2217-2230 | 36 | 2021 |
| Conversational AI: an explication of few-shot learning problem in transformers-based chatbot systems<br>M Ahmed, HU Khan, EU Munir<br>IEEE Transactions on Computational Social Systems 11 (2), 1888-1906 | 24 | 2023 |
| Twitter Bot Detection Using Diverse Content Features and Applying Machine Learning Algorithms<br>FK Alarfaj, H Ahmad, HU Khan, AM Alomair, N Almusallam, M Ahmed<br>Sustainability 15 (6), 1-8 | 22 | 2023 |
| Automated question answering based on improved TF-IDF and cosine similarity<br>M Ahmed, HU Khan, S Iqbal, Q Althebyan<br>2022 ninth international conference on social networks analysis, management ... | 14 | 2022 |
| Context-aware answer selection in community question answering exploiting spatial temporal bidirectional long short-term memory<br>M Ahmed, HU Khan, MA Khan, U Tariq, S Kadry<br>ACM Transactions on Asian and Low-Resource Language Information Processing | 8 | 2023 |
| On solving textual ambiguities and semantic vagueness in MRC based question answering using generative pre-trained transformers<br>M Ahmed, H Khan, T Iqbal, FK Alarfaj, A Alomair, N Almusallam | 7 | 2023 |

### Cited by

| | All | Since 2020 |
|---|---|---|
| Citations | 681 | 674 |
| h-index | 7 | 7 |
| i10-index | 6 | 6 |




### Co-authors

EDIT

- Dr. Hikmat Ullah Khan — Professor/ Chairman, Departmen...
- Fawaz Alarfaj — King Faisal University
- Naif Almusallam — King Faisal University
- Dr. Muhammad Ramzan — University Of Sargodha
- Ehsan Ullah Munir — Professor Department of Comput...
- Saqib Iqbal — Assistant Professor, Software En...
- MOHAMMED AL-OMAIR — king faisal university

https://scholar.google.com.pk/citations?user=wIdOTn8AAAAJ&hl=en

# Lecture Outline



- Why Parallelism Matters Today ?
- Historical Perspective of Parallelism
- Types of Parallelism
- Activity: Identify the Type of Parallelism
- Challenges of Parallelism
- Course Outline
- Material & Resources
- Assessment & Evaluation

# Why Parallelism Matters Today ?



What recent app, tool, or technology have you used that you think requires heavy computation?

# Why Parallelism Matters Today



- **Data Explosion:**
  - We live in an era of billions of connected devices: smartphones, IoT sensors, social media platforms.
  - Example:
    - Facebook generates 4 petabytes of data per day;
    - A single Boeing 787 engine produces 500 GB of data per flight.
  - Managing and processing such vast amounts of data in real time is only possible through parallelism.
- **Artificial Intelligence & Machine Learning**
  - Training modern AI models involves billions of parameters.
  - For example,
    - GPT models are trained on clusters of thousands of GPUs working together.
    - Without parallel computing, AI training would take years instead of weeks/days.

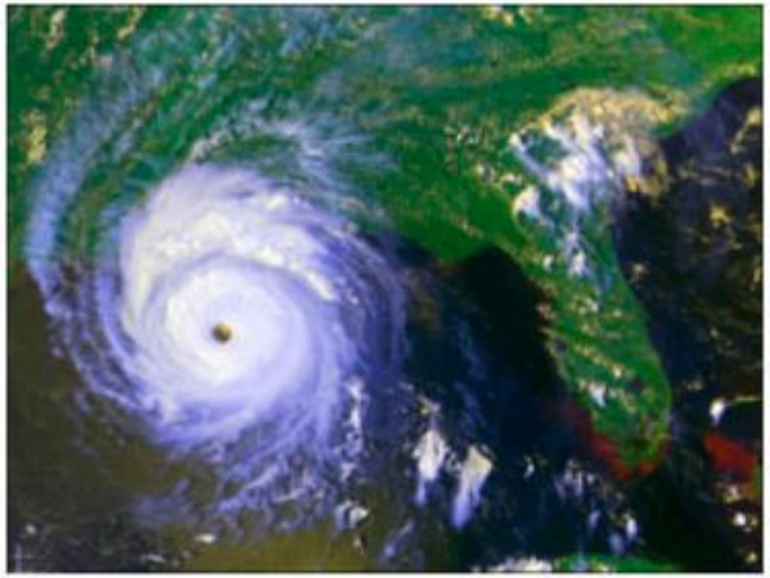
# Why Parallelism Matters Today



- **Graphics & Gaming**
  - Games render **millions of pixels per frame** at 60+ frames per second.
  - GPUs contain **thousands of cores**, each processing a small part of the scene in parallel.
  - Virtual Reality (VR) and Augmented Reality (AR) demand even higher parallel performance.
- **Scientific Simulations**
  - Climate models simulate **decades of atmospheric data** across millions of grid points.
  - DNA sequencing requires **parallel comparison of billions of genetic bases**.
  - These tasks would be **computationally impossible** on single-core processors.
- **Cloud Computing & Big Data**
  - Platforms like Google, Amazon, and Microsoft run on **distributed clusters with millions of servers**.
  - Search queries, recommendations, fraud detection, and streaming are all powered by **parallel/distributed systems**.

Galaxy Formation

Planetary Movments



Climate Change


Rush Hour Traffic

Plate Tectonics

Weather

# Historical Perspective of Parallelism

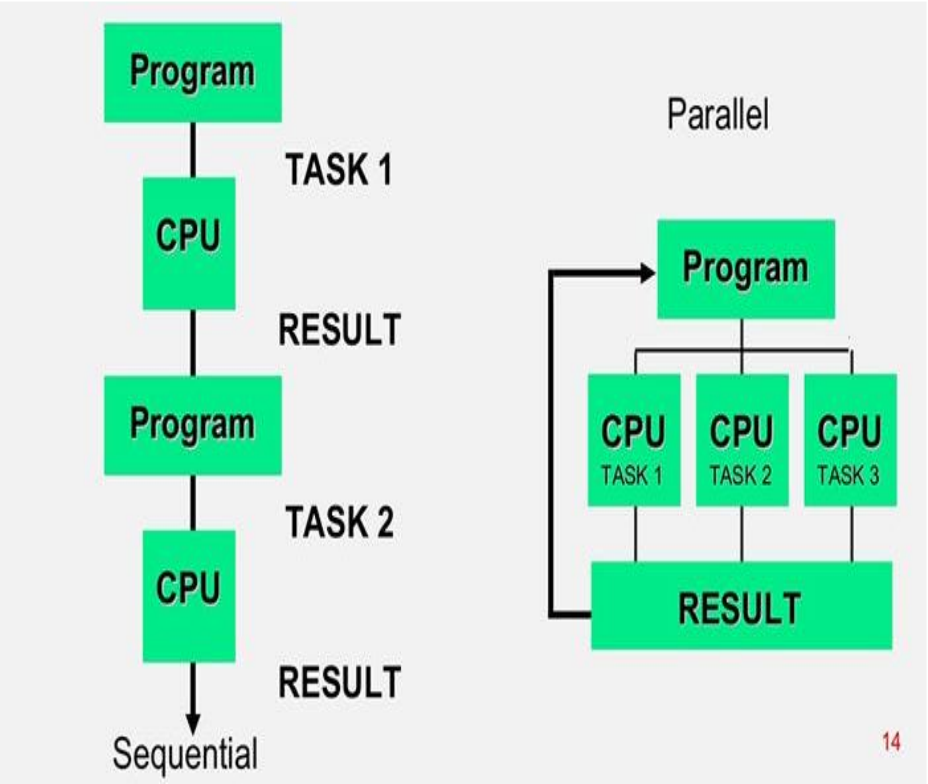


- **1990s -** Multi-core Processors
  - Clock speed scaling slowed due to heat & power limits.
  - Shift to multi-core CPUs (e.g., Intel Pentium Pro, 1995).
  - Brought parallelism into mainstream desktop computing.
- **2000s -** GPUs Beyond Graphics
  - Originally built for gaming and rendering, GPUs proved perfect for massive parallel workloads.
  - NVIDIA introduced CUDA (2006) → unlocked AI, finance, and scientific computing acceleration.
  - Sparked the modern AI revolution.
- **Today** - Cloud-Scale Parallelism
  - Google, AWS, Azure → millions of servers working together.
  - Parallelism powers Google Search, Netflix, YouTube, ChatGPT training.Big Data & AI frameworks rely on distributed clusters.
- **Tomorrow -** Quantum & Neuromorphic (Quantum computing + Neuromorphic chips)

# Types of Parallelism



- **Data Parallelism** → same operation on many data points (e.g., matrix multiplications in AI).
- **Task Parallelism** → different tasks executed simultaneously (e.g., a car's autopilot: vision + planning).
- **Pipeline Parallelism** → stages processed concurrently (e.g., video streaming encoding).



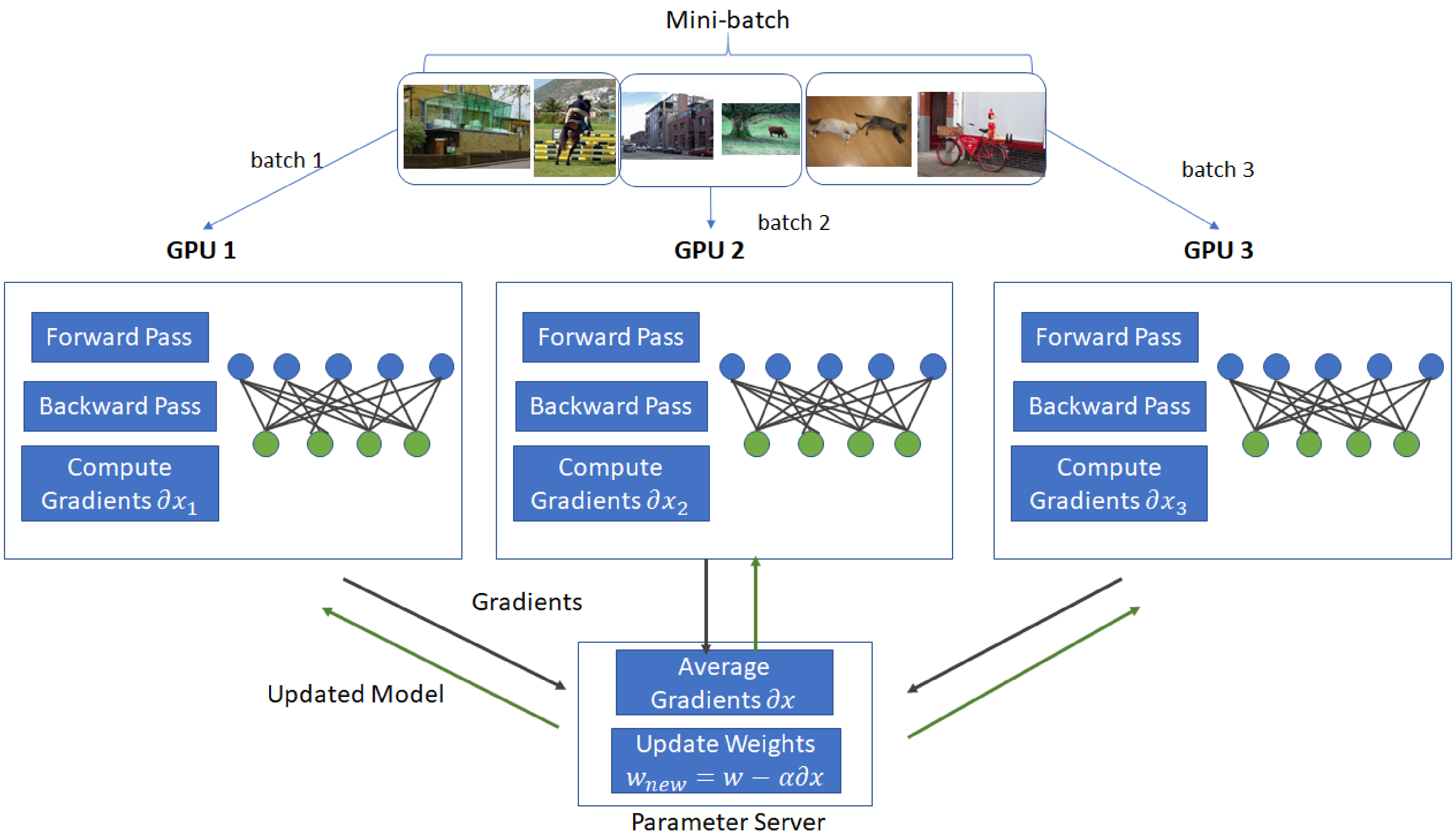
# Data Parallelism



- The *same operation* is applied to *many data items* at the same time.
- **Focus:** *divide the data, keep the operation same.*
- **Examples:**
  - **Artificial Intelligence** → training a neural network: matrix multiplications across millions of weights.
  - **Image Processing** → applying filters (blur, brightness, edge detection) to all pixels in parallel.
  - **Scientific Simulations** → computing forces on particles in physics models simultaneously.
  - **Finance** → calculating risk across thousands of investment portfolios at once.
  - **Weather Forecasting** → updating each cell of an atmospheric grid concurrently.

# Data Parallelism





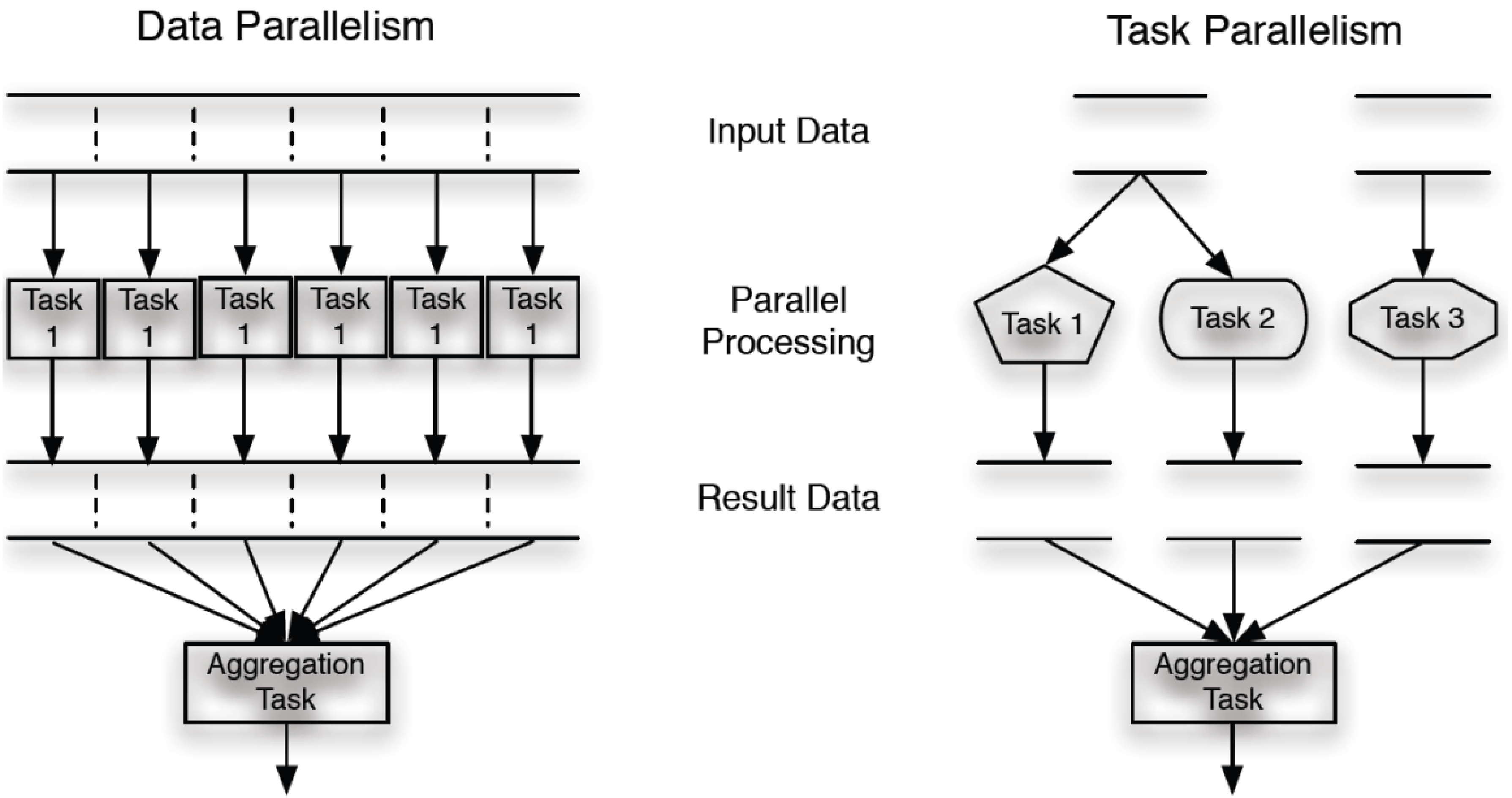
# Task Parallelism



- *Different tasks* (functions, processes) are executed **simultaneously**, often on different processors.
- **Focus:** *divide the tasks, not the data.*
- **Examples:**
  - **Self-Driving Cars →**
    - Vision system detects objects,
    - Navigation plans route,
    - Control adjusts steering → all at once.
  - **Smartphones** → playing music + downloading updates + receiving notifications simultaneously.
  - **Operating Systems** → CPU schedules multiple processes (browser, Word, Spotify) in parallel.
  - **Healthcare Monitoring** → heart rate sensor, oxygen monitor, and alert system all running in real-time.
  - **Cloud Services** → one server handles authentication, another handles data storage, another handles analytics.

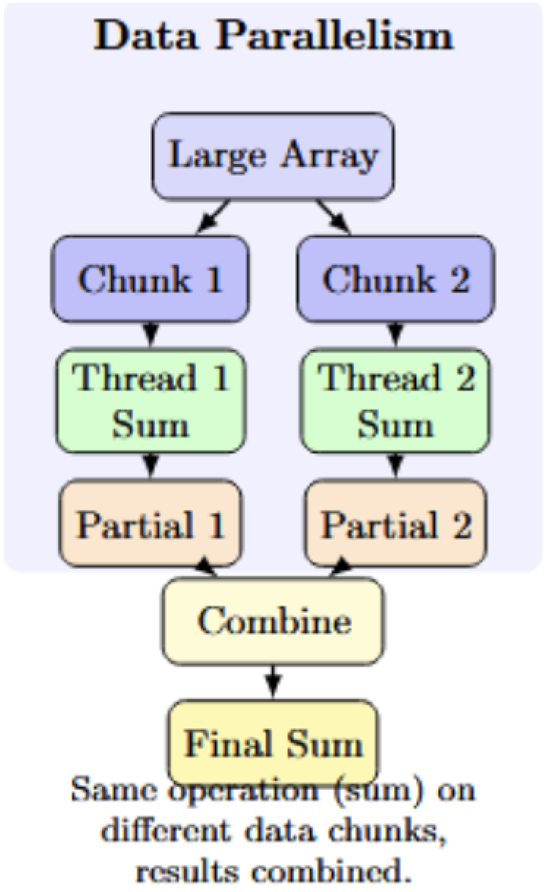
# Data vs Task Parallelism

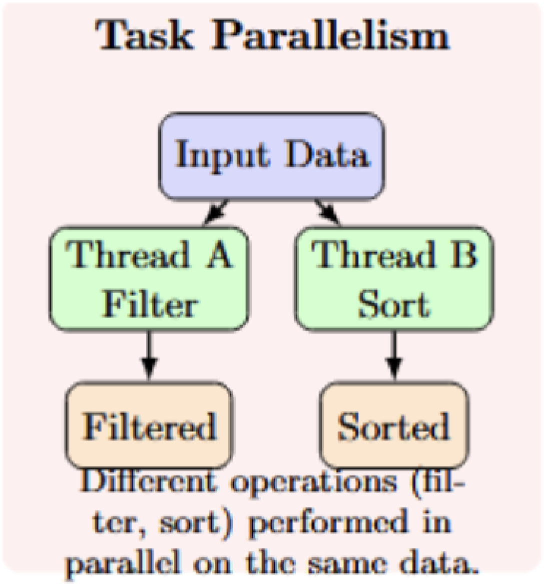




# Data vs Task Parallelism





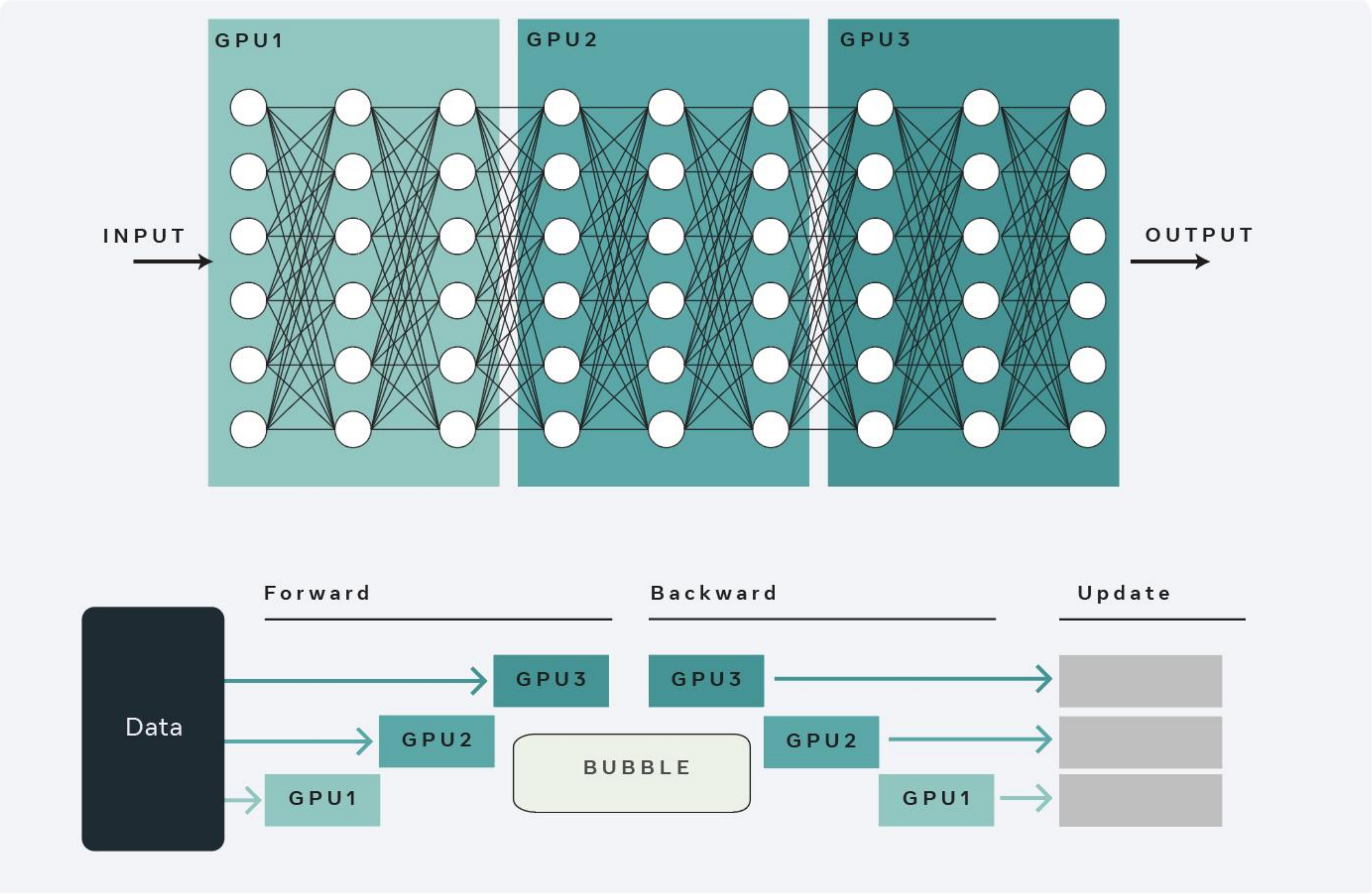
# Pipeline Parallelism



- Work is divided into *stages*, and each stage processes a different part of the task **concurrently**.
- Like an assembly line: one stage finishes its part, while the next begins.
- **Examples:**
  - **Video Streaming (Netflix, YouTube)** → frames compressed → encoded → transmitted → decoded.
  - **Computer Architecture** → CPU instruction pipeline (Fetch → Decode → Execute → Write Back).
  - **Manufacturing Industry** → assembly line where each worker performs a stage in parallel.
  - **Speech Recognition** → audio captured → features extracted → model inference → text output.
  - **Data Analytics Pipelines** → collect → clean → analyze → visualize.

# Pipeline Parallelism





# Activity: Identify the Type of Parallelism



- A weather forecasting model divides the world map into grids, and each processor simulates one region.
- A smartphone runs face recognition while simultaneously streaming music and handling network requests.
- An assembly line in a car factory where one stage installs the engine, another paints, and other tests electronics.

# Challenges of Parallelism



- **Parallelism isn't always easy — common challenges include:**
- **Synchronization** → keeping tasks coordinated.
- **Communication Overhead** → time spent sharing data.
- **Load Balancing** → distributing work fairly.
- **Hardware & Energy Cost** → more resources required.
- **Programming Complexity** → harder to design & debug.

# Challenges of Parallelism



- **Synchronization**
  - Tasks often need shared resources (e.g., memory, files).
  - If not managed, leads to **race conditions** (wrong results).
  - Solutions: locks, semaphores, atomic operations.
    *Example:* Two threads updating a bank account balance.
- **Communication Overhead**
  - Parallel tasks need to exchange results or data.
  - This adds **latency** and reduces efficiency.
  - Especially problematic in **distributed systems** (network delays).
    *Example:* Google Maps servers exchanging traffic data.

# Challenges of Parallelism



- **Load Balancing**
  - Some tasks are heavier than others.
  - Uneven workload = some processors idle while others overwork.
  - Goal: distribute tasks evenly.
    *Example:* Weather simulation where one region (oceans) takes longer than another (desert).
- **Hardware & Energy Cost**
  - More cores/GPUs → higher purchase + electricity costs.
  - Cooling & infrastructure needed at scale (data centers).
  - Efficiency vs. performance trade-off.
    *Example:* Training GPT-4 consumed **thousands of GPUs** for weeks.

# Challenges of Parallelism



- **Programming Complexity**
  - Writing sequential code = simple.
  - Writing parallel code = must handle **synchronization, deadlocks, scalability**.
  - Requires new frameworks (MPI, CUDA, OpenMP).
    *Example:* Debugging a deadlock in a multi-threaded app.

# Course Outline



- Use following link to access course outline:
- https://docs.google.com/document/d/1589Yv9-VyNAsMWaqVSj0XxUvJUZV1_ZU/edit?usp=sharing&ouid=108143013775170108142&rtpof=true&sd=true

# Material & Resources



- **Recommended Books:**
  - An Introduction to Parallel Programming by Peter S. Pacheco, 2nd Edition.
    - https://drive.google.com/file/d/1g04c4SK7vsBx73EAggjQKF4YSAaEy_mO/view?usp=sharing
  - Distributed Systems, Third edition, by Maarten van Steen, Andrew S. Tanenbaum
    - https://drive.google.com/file/d/1mDXba4583u192LxRnyZjp2uRRpWW7OFb/view?usp=sharing

- Lectures and notes will be uploaded on QOBE Portal

# Assessment & Evaluation



- Continuous Assessments (30%)
  - Quizzes (15 marks)
  - Assignments (15 marks)
- Midterm Exam (30 marks)
- Final Examination (40%)

# Thanks